

ABEGPU Kernel 与访存优化：让 GPU 真正忙起来

目 录

1 引言：从“有一堆 kernel”到“GPU 真正忙起来”	2
2 ABEGPU 的源文件结构	2
2.1 device-side kernel 文件	2
2.2 host-side 调度文件	3
3 MPI 通信分支：device 直传与 host 中转	3
4 Kernel 形态与编译参数	3
4.1 grid/block 大小与每线程数据粒度	4
4.2 warp divergence 与寄存器压力	4
4.3 可调的编译参数与 hint	4
5 RDC 与 inline：跨翻译单元的 device 调用	5
5.1 什么时候需要 RDC	5
5.2 *.cuh 与 __device__ __forceinline__	5
6 访存优化：让数据流得起来	5
6.1 global memory 是否 coalesced	5
6.2 重复读取与 shared memory	6
6.3 shared memory 的 padding 与 bank conflict	6
6.4 只读数据与 cache	6
6.5 减少临时数组与写回	6
6.6 数学等价公式的重排	6
7 Kernel 拆分、融合与批处理	7
7.1 拆分巨型 kernel	7
7.2 融合连续不同操作	7
7.3 跨变量批处理相同操作	7
7.4 三类手段的选择标准	8
8 手算例子：拆分前后的总时间估算	8
9 用端到端结果判断优化	8
10 本章你将学会	9
11 要点速查	9
12 小结	9

1 引言：从“有一堆 kernel”到“GPU 真正忙起来”

在 Lab4 的 Task2 中，我们要把 TwoPunctureABE 与 ABEGPU (AMSS-BSSN-Einstein-GPU) 端到端地跑在单卡 V100 上。ABEGPU 不是一个小项目，它的 GPU 化涉及 **BSSN RHS** (BSSN 方程右端项)、**prolongation** (延拓)、**analysis** (分析量计算)、数据打包与 **ghost exchange** (鬼区交换) 等多组 device-side 实现，再加上 host 侧的发射调度，整个程序里有大量 kernel 与复杂控制流。

很多同学一上来就把注意力放在“让某个 kernel 跑得快”。但你很快会发现：把单个 kernel 提速 30%，整程序可能只快了 3%，因为那个 kernel 只占总时间的一小部分，而 GPU 大部分时间在等待数据、等待同步、等待 launch。真正的优化目标是“让 GPU 真正忙起来”，也就是让 SM (Streaming Multiprocessor) 在尽量长的时间窗口里持续有活可干，而不是被同步、被通信、被临时数组写回卡住。

直觉 不妨把 GPU 想象成一条流水线车间。你优化某个工位 (单个 kernel) 的速度，确实能让那个工位的产能上升，但如果上下游工位跟不上、或者传送带 (访存带宽) 堵塞、或者车间主任 (host 调度) 频繁叫停整条线做质检 (全局同步)，车间整体产出并不会按比例提升。真正有效的优化，是看整条流水线的瓶颈在哪里。

本章我们要解决的问题是：给定 ABEGPU 这样一个由数十个 kernel 加上 host 调度组成的大程序，如何系统地分析、改造 kernel 形态与访存行为，使端到端时间下降。我们会先认识 ABEGPU 的源文件结构与通信分支，然后逐层讨论 kernel 编译参数、访存优化、kernel 拆分融合批处理三类不同瓶颈，最后用端到端结果来判断优化是否真的有效。

2 ABEGPU 的源文件结构

要优化一个程序，先要知道它的代码长什么样。ABEGPU 的 GPU 相关代码大致分两类：device-side 的 *.cu 文件实现具体计算 kernel，host-side 的 bssn_step_gpu.C 负责调度。

2.1 device-side kernel 文件

关键的 device kernel 文件如下：

1. bssn_rhs_gpu.cu: 计算 BSSN 方程右端项，是整个演化最重的计算，常常是单文件多阶段的巨型 kernel。
2. diff_new_gpu.cu / lopsidediff_gpu.cu: 实现差分算子，前者用于对称差分，后者用于单侧差分 (边界与某些 upwind 场景)。
3. kodiss_gpu.cu: 实现 **Kreiss-Olger dissipation** (Kreiss-Olger 耗散)，用来抑制数值振荡。
4. rungekutta4_rout_gpu.cu: 实现 **Runge-Kutta** (龙格库塔) 时间积分，通常四阶 RK 会调用 RHS 多次。
5. prolongrestrict_cell_gpu.cu: 实现 **AMR** (自适应网格加密) 的 prolong 与 restrict 操作。
6. MPatch_gpu.cu / Parallel_GPU.cpp: 管理多 patch 的数据与通信。
7. surface_integral_gpu.cu / getnp4_gpu.cu / fadmquantites_bssn_gpu.cu: 分析量计算，例如表面积分、ADM 质量、FADM 量等。

看到这些文件名你就能感觉到，ABEGPU 把“演化核心”与“分析量”分别放在不同 kernel 里。这意味着演化与分析在原则上是可以并行或错开的，这是后续 stream 重叠优化的基础。

2.2 host-side 调度文件

bssn_step_gpu.C 是 host 侧的 GPU 调度中枢，它负责的事情包括：kernel 发射、RK 子步推进、必要的同步、ghost zone 的交换触发。一个 RK 时间步里，host 会按顺序发射若干 RHS kernel，再做 ghost exchange，再进入下一个子步。

直觉 host 调度像乐队的指挥，kernel 像乐手。指挥要做的事情不是替乐手吹奏，而是决定“谁先吹、什么时候吹、吹完一组要不要等齐”。如果指挥每个小节都让全员停下来等齐再继续，那整场演出的节奏会被同步拖慢；反过来如果指挥在可以并行的地方合理安排，就能让计算与通信、计算与分析错开进行。

理解源文件结构后我们就明白：优化 ABEGPU 不只是优化 bssn_rhs_gpu.cu，还要看 host 调度是否制造了不必要的等待，是否把本可以并行的部分串行了。

3 MPI 通信分支：device 直传与 host 中转

ABEGPU 在 ghost exchange 时要把 patch 边界数据从一个 MPI rank 传给另一个。这里存在两条通信路径：

1. **CUDA-aware MPI**: MPI 实现可以直接接收 device 指针，通信时数据不必先拷回 host。这种路径少了 D2H（device 到 host）与 H2D（host 到 device）两次拷贝，但需要 MPI 实现被构建为支持 CUDA-aware。
2. **host staging**: 先把 device 上的 ghost 数据 D2H 拷到 host 缓冲，再用普通 MPI 发送，接收端再 H2D 拷回 device。这种方式兼容任何 MPI，但每次通信多两次拷贝。

在 macrodef.h 中，宏 MPI_CUDA_AWARE 默认被设为 0，也就是说默认走 host staging 路径。这意味着即便你的 MPI 实现支持 CUDA-aware，程序默认也不会用它。

这是一个很容易踩的坑：你以为程序“已经在用 GPU 直传”，其实它走的是 host 中转。要确认当前到底走哪条路径，去看 macrodef.h 里 MPI_CUDA_AWARE 的值，并在通信热点处加时间统计。

什么时候 host staging 反而可接受？当通信量很小、或 GPU 计算时间远长于通信时间时，D2H/H2D 的开销可能被计算掩盖。但当 patch 边界较大、ghost exchange 频繁时，这两次拷贝会成为瓶颈，这时就要考虑启用 CUDA-aware 路径（这部分在下一章 stream 与多卡里展开）。

4 Kernel 形态与编译参数

有了源文件全景，我们进入第一个优化层面：kernel 自身的形态与编译选项。这一层的核心问题是，每个 kernel 的 grid/block 大小、每线程数据粒度、空闲线程比例、warp divergence、寄存器压力，是否被合理设置。

4.1 grid/block 大小与每线程数据粒度

一个 kernel 的吞吐，首先取决于它每次 launch 能让多少 thread 真正干活。常见的低效模式有：

1. block 大小不是 32 的整数倍，导致最后一个 warp 残缺，部分 lane 空闲。
2. grid 维度没有覆盖到全部计算单元，部分 SM 上没有活可干。
3. 每个线程只处理一个点，而该点上的计算其实很轻，导致 launch 与访存开销占比过高。
4. 每个线程处理过多点，寄存器压力上升，occupancy 下降。

直觉 把 grid/block 想成排兵布阵。block 太小，兵太少，每个军官（warp scheduler）带不满一队，后勤（寄存器与共享内存）摊不薄；block 太大，单兵扛的活太多，装备（寄存器）超重，反而上不了前线。合适的 block 大小通常是 128 或 256，再配合每线程处理 1 到若干个点的`数据粒度`，做实际对比。

4.2 warp divergence 与寄存器压力

warp divergence（分支发散）指同一 warp 内不同 thread 走不同分支，导致分支被串行执行。在差分算子中，内部点与边界点的处理逻辑不同，如果不做分 kernel 或分支对齐，就容易产生 divergence。

寄存器压力则是另一个常见瓶颈。一个 kernel 用的寄存器越多，每个 SM 能同时驻留的 warp 就越少，**occupancy**（占用率）就越低。如果 kernel 里临时变量太多、循环展开过度，寄存器就会爆。

4.3 可调的编译参数与 hint

CUDA 提供了若干编译期与源码内的 hint 来控制这些行为：

1. `__forceinline__`：对小型频繁调用的辅助函数（例如插值权重、坐标变换）加这个修饰，让编译器在调用处展开，省去函数调用开销，但会增大代码体积。
2. `__launch_bounds__(maxThreadsPerBlock, minBlocksPerSM)`：显式告诉编译器每个 block 最多多少线程、每个 SM 至少要驻留多少 block，编译器据此调整寄存器分配，在寄存器与 occupancy 之间做权衡。
3. `#pragma unroll N`：对小规模固定次数的循环做展开，减少循环控制开销，但展开过度会增加寄存器压力与代码体积。
4. nvcc 编译参数：例如 `-maxrregcount` 限制每线程最大寄存器数，`-O3`、`--use_fast_math` 等会影响生成代码。

有一条原则很重要：不要为了让“kernel 数量变少”而把一个本来可以独立调优的 kernel 合并成不可调优的巨型 kernel。减少 launch 数量是好目标，但前提是合并后的 kernel 仍然可被 `ncu` 分析、可被 `hint` 引导、可被拆回来。

5 RDC 与 inline: 跨翻译单元的 device 调用

ABEGPU 在构建时启用了 `-rdc=true` 与 `CUDA_SEPARABLE_COMPILATION`, 也就是 **Relocatable Device Code** (可重定位设备代码)。这意味着不同翻译单元 (TU) 之间的 `__device__` 函数可以在链接期被解析, 允许跨 TU 的 device 调用。

5.1 什么时候需要 RDC

如果你的 kernel A 调用了在另一个 .cu 文件里定义的 `__device__` 函数 B, 且两者没有被 `inline` 进同一个 TU, 就需要 RDC 来在链接期把它们接上。RDC 的好处是代码组织清晰、可复用; 代价是会有跨 TU 调用的运行时开销, 且某些全局优化 (例如激进 `inline`) 会被阻碍。

直觉 RDC 像是把多个 .cu 文件编成一本“可链接的设备代码合集”。没有 RDC 时, 每个文件各自为政, 跨文件调用要么靠 `inline` 在编译期抹平, 要么根本无法实现; 有了 RDC, 跨文件调用在链接期可以接上, 但每次调用要查一次函数表, 比 `inline` 慢。

5.2 *.cuh 与 `__device__ __forceinline__`

对小型频繁调用的 device 函数, 常见做法是把它的实现放进 *.cuh 头文件, 并加上 `__device__ __forceinline__`。这样每个调用点在编译期就把函数体展开, 省去跨 TU 的运行时代价, 相当于编译期 `inline`。

这种做法的代价是:

1. 代码体积增大, 因为每个调用点都嵌了一份函数体, 可能导致 **icache** (指令缓存) 压力。
2. 如果 `inline` 后函数里临时变量多, 可能推高调用 kernel 的寄存器使用, 降低 **occupancy**。
3. 对真正大型、调用次数少的 device 函数, `inline` 反而不划算。

判断要不要 `inline` 的经验: 函数体短、被高频调用、且不含大量临时变量, 就 `inline`; 函数体长、调用次数少、或本身用很多寄存器, 就保留普通 `__device__` 函数并依赖 RDC 链接。

6 访存优化: 让数据流得起来

kernel 形态决定了“计算能跑多快”, 而访存决定了“数据供得上吗”。ABEGPU 的大量 kernel 是 stencil 类算子 (差分、耗散、RHS), 它们的共同特征是: 每个点要读取自己与若干邻居的数据, 访存模式高度依赖邻域。

6.1 global memory 是否 coalesced

coalesced access (合并访问) 指同一 warp 内相邻 thread 访问相邻地址, 这样硬件可以把多次访问合并成少数几次事务。如果 thread 访问的是分散地址, 每次访问都会浪费带宽。在 stencil 算子里, 内存布局 (例如 SoA 还是 AoS) 直接决定能否 coalesced。

直觉 把 global memory 想成一条很宽的高速公路，每个 warp 一次能拉一整车数据。coalesced 就是车上每个座位都坐了同 warp 里的人，一次拉走；非 coalesced 就是车上大部分座位空着，却要跑很多趟。

6.2 重复读取与 shared memory

stencil 算子的另一个特征是邻域数据被多个 thread 重复读取。例如一个 7 点 stencil，中心点会被自己和周围 6 个邻居各读一次。如果都从 global memory 读，等于每个点被读了 7 次。这种场景正是 **shared memory**（共享内存）的用武之地：把一个 tile 的数据一次性从 global 加载到 shared，然后 block 内所有 thread 从 shared 重复读取，速度高一个数量级。

6.3 shared memory 的 padding 与 bank conflict

shared memory 被分成 32 个 bank，如果同一 warp 内多个 thread 访问同一 bank，就会产生 **bank conflict**（bank 冲突），访问被串行化。常见的解决办法是对 tile 做一点 **padding**（填充），让访问模式错开 bank，同时 padding 也能改善对齐。

6.4 只读数据与 cache

对于在整个 kernel 期间只读不改的数据（例如演化系数、坐标度量），可以用 `__ldg()` 或 `const __restrict__` 修饰，让硬件走 **read-only cache**（只读缓存，即 texture/L1 路径），减少对普通 L1 与 global 带宽的争用。

6.5 减少临时数组与写回

一个隐蔽但很重要的优化是：尽量减少临时数组在 global memory 上的写回。如果一个中间量只在下一步被消费，与其写回 global 再读回来，不如保留在寄存器或 shared memory 里直接传给下一步。这等价于把两个 kernel 在数据维度上做生产者消费者连接。

6.6 数学等价公式的重排

有时候数学上等价的两种写法，访存代价差别很大。例如把“先算 A 再算 B 再合并”改成“按点合并算 A 与 B”，可能减少中间结果的总写入量。这种重排要保证数学等价性，并配合正确性校验。

例 假设一个 stencil kernel 每个点要读 7 个邻居值，每次邻居值 8 字节，单卡 V100 的 global 带宽约 900 GB/s。如果一个点被直接从 global 重复读 7 次，那么每点要读 $7 \times 8 = 56$ 字节。设网格有 $N^3 = 256^3 \approx 1.68 \times 10^7$ 个点，单步读取 $\approx 9.4 \times 10^8$ 字节 ≈ 0.94 GB，单步只读就要约 $\frac{0.94}{900} \times 10^3 \approx 1.04$ ms。换算到一次演化 1000 步，光读就要 1 秒以上，且这还没算写。若改用 shared memory 把每点邻居一次性加载，则每点只需从 global 读约 1 次自己，总读 $\approx 1.68 \times 10^7 \times 8 \approx 1.34 \times 10^8$ 字节 ≈ 0.13 GB，单步读约 $\frac{0.13}{900} \times 10^3 \approx 0.15$ ms，提速约 7 倍。这就是 stencil 访存优化的数量级。

7 Kernel 拆分、融合与批处理

这一层我们要面对三种不同性质的瓶颈，对应三类不同的改造手段。它们看起来互相对立（拆分 vs 融合），但其实针对的是不同的病根。

7.1 拆分巨型 kernel

bssn_rhs_gpu.cu 里的 RHS 往往是多阶段计算（例如先做差分，再做耗散，再做源项），有时被写成一个巨型 kernel。巨型 kernel 的好处是 launch 少、中间结果可以留在寄存器里不写回；坏处是寄存器压力大、occupancy 低、icache 膨胀，ncu 里能看到寄存器数很高、活跃 warps 比例低。

如果 ncu 显示这些症状，就该考虑按数据依赖把巨型 kernel 拆成几个小 kernel。拆分后每个 kernel 寄存器少、occupancy 高，但代价是增加了 launch 数量、以及阶段之间要把中间结果写回 global 再读回。所以拆分不是无脑拆，要看寄存器收益是否大于额外的 global 往返。

直觉 巨型 kernel 像一个人扛三份活，省了交接时间但累得走不动；拆分后像三个人各扛一份，交接要花时间但每个人轻装上阵跑得快。能不能拆、拆完划不划算，取决于“交接成本”（阶段间 global 往返 + launch）与“轻装收益”（occupancy 提升）谁大。

7.2 融合连续不同操作

与拆分相反的情形是：两个连续 kernel 满足以下条件时，可以融合：

1. 它们遍历同一个迭代域（例如都对整个内部网格做一遍）。
2. 第二个 kernel 的大部分输入正好是第一个 kernel 的输出（生产者消费者关系）。

融合的好处是：减少一次 launch、减少一次全局同步、减少中间数组在 global 上的往返。坏处是：融合后 kernel 的寄存器生命周期被拉长（第一个 op 的寄存器要一直活到第二个 op 用完），可能降低 occupancy；原本可以并行的两个 op 被串行化在同一个 kernel 内；边界处理、ghost exchange、RK 阶段间的真实依赖不能跳过。

这里有一个常见误区：以为“融合总是更好”。其实融合把两个 op 串行了，如果它们本可以放在不同 stream 上重叠，融合反而损失了并行度。所以融合与否要看 ncu 与 nvprof 的端到端数据，不能拍脑袋。

7.3 跨变量批处理相同操作

ABEGPU 的演化里有大量“对每个状态变量做同一种操作”的场景。例如 bssn_step_gpu.C 会遍历所有状态变量，逐个调用 RK 更新 wrapper，每次都有一次 launch。这种“对同一组数据反复做同一件事”的场景，适合做 **batch**（批处理）：给 kernel 加一个变量维度，让一次 launch 处理多个变量。

批处理的好处是 launch 数量按变量数成倍下降。但要小心几件事：

1. 每个变量的属性（边界条件、ghost 宽度、是否需要特殊处理）必须保留，不能为了批处理而丢掉。
2. 多变量合并后，每个变量在内存里可能不是连续的，要检查指针间接访问是否会破坏 coalesced。

3. 变量之间的真实依赖（例如一个变量是另一个变量的导数）不能被批处理打乱顺序。

直觉 批处理像把“一个工人依次做十个零件”改成“一台机器同时做十个零件”。机器快，但前提是这十个零件的工艺要求一致，且它们之间没有先后依赖。

7.4 三类手段的选择标准

把三种手段放在一张表里对比：

手段	适用瓶颈	主要收益	主要代价
拆分巨型 kernel	寄存器压力、occupancy 低、icache 膨胀	occupancy 上升、寄存器下降	launch 增加、阶段间 global 往返
融合连续 op	launch 多、中间数组往返、sync 多	launch 减少、中间写回减少	寄存器生命周期延长、串行化
跨变量批处理	同一 op 对多变量反复 launch	launch 成倍减少	指针间接、coalesced 风险、依赖打乱

三类手段对应三类不同病根。判断你的程序病在哪儿，要靠 *ncu* 的指标：*launch* 数量、寄存器数、*occupancy*、内存流量、*stall* 原因。先诊断后下药。

8 手算例子：拆分前后的总时间估算

例 设单卡 V100 的巨型 RHS kernel 每线程用 120 个寄存器，对应 occupancy 50%。一次演化步内该 kernel 耗时 $T_{\text{big}} = 8.0 \text{ ms}$ 。把它拆成两个 kernel，各用 60 个寄存器，occupancy 升到 75%。假设计算量不变，纯计算时间与 occupancy 大致成反比（实际还要看 warp 调度，这里简化估算），则每个小 kernel 的纯计算时间约为 $\frac{8.0}{2} \times \left(50\% \frac{75}{75}\right) \approx 2.67 \text{ ms}$ ，两个共 5.34 ms。但拆分后多了一次中间结果写回与读取：设中间数组 $M = 2 \times 10^8$ 字节，V100 带宽 900 GB/s，往返一次约 $2 \times \frac{2 \times 10^8}{9 \times 10^{11}} \times 10^3 \approx 0.44 \text{ ms}$ ，再算上一次额外 launch 约 0.01 ms。总时间 $\approx 5.34 + 0.45 \approx 5.79 \text{ ms}$ ，相比原来 8.0 ms 节省约 2.21 ms，加速比约 1.38 ×。若中间数组更大或带宽更紧张，拆分的收益就会被往返吃掉，这正是为什么要用端到端结果判断。

这个例子说明了三件事：拆分能提升 occupancy，但不是免费的；中间数组的 global 往返是拆分的主要代价；最终判断要看端到端时间，而不是某个 kernel 的局部指标。

9 用端到端结果判断优化

我们反复强调“用端到端结果判断”，这一节把这件事说清楚。一个优化是否有效，最终要看的是整程序从启动到结束的时间，以及在这段时间里 GPU 是否真的在忙。具体要看这些指标：

1. **launch 数量**：整程序总共发了多少个 kernel launch，是否被减少。

2. **GPU 活跃时间**: 用 ncu 或 nvprof 看 GPU 在整段时间里有多少比例在执行 kernel, 多少在空闲等待。
3. **寄存器与 occupancy**: 每个关键 kernel 的寄存器数与实际 occupancy。
4. **内存流量**: global memory 的实际吞吐是否接近带宽上限, 还是被低效访问浪费。
5. **同步等待**: cudaDeviceSynchronize、cudaStreamSynchronize 等同步点的等待时间。
6. **完整端到端时间**: 从程序启动到写出结果文件的总时间。
7. **正确性**: 优化后结果是否仍满足精度要求。

fusion 与 *fission* 都不是默认正确的。一个 *fusion* 在 A 程序上有效, 在 B 程序上可能因为 *occupancy* 变化而失效。永远拿端到端数据说话。

10 本章你将学会

1. 分析 ABEGPU 中 kernel 的形态与编译参数, 判断 grid/block、寄存器、occupancy 是否合理。
2. 设计访存优化, 包括 coalesced、shared memory、padding、只读 cache、减少临时数组写回、数学等价重排。
3. 决定 kernel 的拆分、融合与批处理, 理解三类手段各自的适用瓶颈与代价。
4. 用端到端结果 (launch 数、GPU 活跃时间、寄存器、occupancy、内存流量、同步等待、完整时间、正确性) 判断优化是否有效, 而不是看单 kernel 局部指标。

11 要点速查

主题	要点
源文件结构	device *.cu 实现 kernel, bssn_step_gpu.C 负责 host 调度
MPI 通信分支	CUDA-aware 直传 vs host staging, MPI_CUDA_AWARE 默认 0
kernel 形态	检查 grid/block、每线程粒度、空闲线程、warp divergence、寄存器压力
编译 hint	__forceinline__、__launch_bounds__、#pragma unroll、nvcc 参数
RDC	-rdc=true 启用跨 TU device 调用, 小型函数用 .cuh + __device__ __forceinline__
访存优化	coalesced、shared memory、padding、只读 cache、减临时数组、公式重排
拆分巨型 kernel	occupancy 上升 vs 阶段间 global 往返, 靠 ncu 诊断
融合连续 op	减 launch 与中间写回 vs 寄存器延长与串行化
跨变量批处理	launch 成倍减少 vs 指针间接、coalesced、依赖打乱风险
判断标准	端到端时间、GPU 活跃时间、occupancy、内存流量、正确性

12 小结

本章我们围绕“让 GPU 真正忙起来”这个目标, 把 ABEGPU 的 kernel 与访存优化拆成了几个层次。先认识了源文件结构与 MPI 通信分支, 知道默认走 host staging; 再讨论了 kernel 形态与编译参数, 包括 grid/block、寄存器、occupancy 以及 __forceinline__、__launch_bounds__、RDC 等工具; 接着深入访存优化, 强调 coalesced、shared memory、

padding 与减少临时数组写回；最后用三类不同瓶颈对应拆分、融合、批处理三类手段，并通过手算例子说明为什么必须用端到端结果判断。下一章我们会把视线从单 kernel 扩展到单卡上的 stream 异步执行与双卡并行，看通信与计算如何重叠。

讲义基于 HPC101 2025 Lab4 实验内容编写